# SISTEMA DE GESTIÓN DE BIBLIOTECA

## Programación I - Proyecto Integrador

**UNIVERSIDAD ARGENTINA DE LA EMPRESA (UADE)**
**Materia:** Programación I
**Profesor:** ABEL ISRAEL LAIME HUANCA
**Grupo:** 5

## INTEGRANTES DEL EQUIPO

| **Apellido y Nombre** | **Rol** |
|---|---|
| **TASSONE, FACUNDO JAVIER** | Líder de Proyecto + Menú Principal |
| **IGLESIAS, MARTIN DAMIAN** | Especialista en Persistencia de Datos |
| **MILITELLO, LUCA SANTINO** | Desarrollador Módulo Libros |
| **LEE, HYO LIN** | Desarrollador Módulo Usuarios |
| **KIDYBA, LAUTARO AGUSTIN** | Desarrollador Módulo Préstamos |

# ÍNDICE


1. Idea General del Proyecto
2. Definición de Roles del Equipo
3. Primer Boceto del Menú Principal
4. Ejemplo Inicial de Entrada/Salida por Consola
5. Estructura Técnica Inicial
6. Actualizaciones del Proyecto
7. Documentación Técnica del código
8. Validaciones adicionales implementadas
9. Capturas del sistema funcionando

# 1. IDEA GENERAL DEL PROYECTO

## Nombre del Proyecto

Sistema de Gestión de Biblioteca

## Descripción del Proyecto

Desarrollo de una aplicación de consola en Python que simule las operaciones básicas de una biblioteca. El sistema permitirá gestionar libros, usuarios y préstamos de manera sencilla e intuitiva.

## Objetivo

Crear un sistema funcional que permita:

- **Gestión de Libros**: Agregar, buscar, modificar y eliminar libros del catálogo
- **Gestión de Usuarios**: Registrar socios, mantener sus datos actualizados
- **Sistema de Préstamos**: Controlar préstamos y devoluciones de libros

## Alcance

- **Lenguaje**: Python
- **Interfaz**: Consola/Terminal
- **Persistencia**: Archivos JSON

---

# 2. DEFINICIÓN DE ROLES DEL EQUIPO

| **Integrante** | **Rol** | **Responsabilidades Principales** |
|---|---|---|
| TASSONE, FACUNDO JAVIER | **Líder de Proyecto + Menú Principal** | Coordinación del equipo, seguimiento del cronograma, implementación del menú principal y sistema de navegación |
| IGLESIAS, MARTIN DAMIAN | **Especialista en Persistencia de Datos** | Diseño de la estructura de datos, implementación de la persistencia en archivos JSON, funciones de carga y guardado |

| Integrante | Rol | Responsabilidades Principales |
|---|---|---|
| MILITELLO, LUCA SANTINO | **Desarrollador Módulo Libros** | CRUD completo de libros: agregar, buscar, listar, modificar y eliminar libros del sistema |
| LEE, HYO LIN | **Desarrollador Módulo Usuarios** | CRUD completo de usuarios: registro de socios, búsqueda, actualización y gestión de usuarios |
| KIDYBA, LAUTARO AGUSTIN | **Desarrollador Módulo Préstamos** | Sistema de préstamos y devoluciones, validaciones de disponibilidad, control de fechas |

### Coordinación entre Módulos

- Cada desarrollador trabajará en su módulo específico
- El especialista en datos dará soporte a todos los módulos para la persistencia
- El líder coordinará la integración de todos los módulos

---

# 3. PRIMER BOCETO DEL MENÚ PRINCIPAL

### Estructura Principal

```
═══════════════════════════════════════

  SISTEMA DE GESTIÓN DE BIBLIOTECA

═══════════════════════════════════════

1. GESTIÓN DE LIBROS

   1.1. Agregar libro
   1.2. Buscar libro
   1.3. Listar todos los libros
   1.4. Modificar libro
   1.5. Eliminar libro
   1.6. Volver al menú principal

2. GESTIÓN DE USUARIOS
```

2.1. Registrar nuevo usuario
2.2. Buscar usuario
2.3. Listar usuarios
2.4. Modificar datos de usuario
2.5. Eliminar usuario
2.6. Volver al menú principal

3. SISTEMA DE PRÉSTAMOS

3.1. Realizar préstamo
3.2. Registrar devolución
3.3. Ver préstamos activos
3.4. Historial de préstamos
3.5. Volver al menú principal

4. SALIR DEL SISTEMA

═══════════════════════════════════════

Seleccione una opción [1-4]: _

### Flujo de Navegación

INICIO → MENÚ PRINCIPAL → SUBMENÚ ESPECÍFICO → OPERACIÓN → CONFIRMACIÓN → MENÚ PRINCIPAL

### Validaciones del Menú

- Control de opciones válidas (1-4)
- Mensajes de error para entradas inválidas
- Confirmación antes de salir del sistema
- Navegación intuitiva entre módulos

---

## 4. EJEMPLO INICIAL DE ENTRADA/SALIDA POR CONSOLA

### Ejemplo 1: Inicio del Sistema

═══════════════════════════════════════

SISTEMA DE GESTIÓN DE BIBLIOTECA

═══════════════════════════════════════

Versión 1.0 - Grupo 5

Inicializando sistema...

✓ Cargando datos de libros...
✓ Cargando datos de usuarios...
✓ Cargando datos de préstamos...

¡Sistema listo para usar!

Presione ENTER para continuar...

## Ejemplo 2: Agregar un Libro (Módulo Libros)

═══════════════════════════════════════

AGREGAR NUEVO LIBRO

═══════════════════════════════════════

Ingrese los datos del libro:

Título: El Principito
Autor: Antoine de Saint-Exupéry
ISBN: 978-84-376-0494-7
Año de publicación: 1943
Género: Literatura infantil
Cantidad de ejemplares: 5

─────────────────────────────────────

CONFIRMACIÓN DE DATOS:

─────────────────────────────────────

Título: El Principito
Autor: Antoine de Saint-Exupéry
ISBN: 978-84-376-0494-7
Año: 1943
Género: Literatura infantil
Ejemplares: 5

─────────────────────────────────────

¿Los datos son correctos? (s/n): s

✓ ¡Libro agregado exitosamente!

ID asignado: LIB001

Presione ENTER para continuar...

## Ejemplo 3: Realizar Préstamo (Módulo Préstamos)

```
═══════════════════════════════════════════════════

        REALIZAR PRÉSTAMO

═══════════════════════════════════════════════════

Paso 1: Identificar al usuario

DNI del usuario: 12345678

✓ Usuario encontrado:

  Nombre: Juan Pérez
  DNI: 12345678
  Estado: Activo

Paso 2: Seleccionar libro

ID del libro: LIB001

✓ Libro encontrado:

  Título: El Principito
  Autor: Antoine de Saint-Exupéry
  Disponibles: 4 de 5

───────────────────────────────────────────────────

RESUMEN DEL PRÉSTAMO:

───────────────────────────────────────────────────

Usuario: Juan Pérez (DNI: 12345678)
Libro: El Principito
Fecha de préstamo: 22/09/2025
Fecha de devolución: 06/10/2025 (14 días)

───────────────────────────────────────────────────

¿Confirma el préstamo? (s/n): s
```

✓ ¡Préstamo registrado exitosamente!

Código: PREST001

El libro debe devolverse antes del: 06/10/2025

Presione ENTER para continuar...

## Ejemplo 4: Manejo de Errores

═══════════════════════════════════════

BUSCAR LIBRO

═══════════════════════════════════════

Ingrese el título a buscar:

Error: El título no puede estar vacío.

Ingrese el título a buscar: python

Buscando libros con título "python"...

No se encontraron libros con ese título.

¿Desea realizar otra búsqueda? (s/n): n

Regresando al menú principal...

---

# 5. ESTRUCTURA TÉCNICA INICIAL

## Archivos del Proyecto

sistema-biblioteca/

```
├── main.py                  # Archivo principal con menú
├── modulos/
│   ├── gestion_libros.py     # Módulo de libros
│   ├── gestion_usuarios.py   # Módulo de usuarios
│   ├── gestion_prestamos.py  # Módulo de préstamos
│   └── persistencia.py       # Manejo de archivos JSON
├── datos/
│   ├── libros.json          # Datos de libros
│   ├── usuarios.json        # Datos de usuarios
│   └── prestamos.json       # Datos de préstamos
└── utilidades.py            # Funciones auxiliares
```

## Tecnologías a Utilizar

- **Python 3.x**: Lenguaje principal
- **JSON**: Formato de persistencia de datos
- **Librerías estándar**: `datetime`, `json`, `os`

## Estructura de Datos (JSON)

**Libro:**

```
{
  "id": "LIB001",
  "titulo": "El Principito",
  "autor": "Antoine de Saint-Exupéry",
  "isbn": "978-84-376-0494-7",
  "año": 1943,
  "genero": "Literatura infantil",
  "total_ejemplares": 5,
  "ejemplares_disponibles": 4
}
```

**Usuario:**

```
{
  "dni": "12345678",
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan.perez@email.com",
  "telefono": "1234567890",
  "fecha_registro": "22/09/2025",
```

```
  "activo": true
}
```

**Préstamo:**

```
{
  "codigo": "PREST001",
  "usuario_dni": "12345678",
  "libro_id": "LIB001",
  "fecha_prestamo": "22/09/2025",
  "fecha_devolucion_esperada": "06/10/2025",
  "fecha_devolucion_real": null,
  "estado": "activo"
}
```

# 6. ACTUALIZACIONES DEL PROYECTO

## ENTREGA 2 - Implementación (04/10/2025)

### Código Implementado:

- **main.py**: Sistema de menú principal funcional con navegación completa
- **Módulos operativos**: Cada módulo (libros, usuarios, préstamos) con funcionalidades básicas implementadas
- **Sistema de utilidades**: Funciones auxiliares para limpieza de pantalla y pausa

### Funcionalidades Activas:

- **Gestión de Libros**: Agregar y listar libros con almacenamiento en memoria
- **Gestión de Usuarios**: Registrar y listar usuarios con datos básicos (nombre, apellido, DNI)
- **Sistema de Préstamos**: Crear y listar préstamos con ID automático y estado
- **Navegación completa**: Menús y submenús completamente funcionales

### Estructura de Datos Implementada:

```
# Listas en memoria para almacenamiento temporal

libros = []      # Diccionarios con id, titulo, autor
usuarios = []    # Diccionarios con nombre, apellido, dni
prestamos = []   # Diccionarios con id, usuario_dni, libro_titulo, estado
```

### Validaciones Implementadas:

- Control de opciones válidas en todos los menús
- Manejo de errores de entrada del usuario
- Navegación segura entre módulos

### Estado Actual del Proyecto:

- Menú funcionando con opciones básicas
- Estructura de directorios del proyecto
- Funciones principales esbozadas
- Uso de listas y cadenas (strings)

## Próximos Pasos (Entrega 3):

- Desarrollo de estructura de datos completa
- Implementar persistencia real con archivos JSON
- Agregar validaciones más robustas
- Funciones de búsqueda y modificación
- Integración completa entre módulos
- Sistema de préstamos con validación de disponibilidad

---

# 7. Documentación Técnica del codigo

## 7.1 Módulo de Persistencia (persistencia.py)

Este es el núcleo del sistema de datos, proporcionando operaciones atómicas y seguras:

### Funciones Principales:

- `leer(tabla)`: Carga datos desde archivos JSON de forma segura
- `escribir(tabla, registros)`: Guarda datos con escritura atómica
- `agregar_registro(tabla, nuevo_registro)`: Añade nuevos registros
- `eliminar_registro(tabla, predicado)`: Elimina registros por condición
- `generar_id(prefijo, registros, campo_id)`: Genera IDs únicos automáticamente

```python
def leer(tabla: Tabla) -> List[Dict[str, Any]]:
    """Lee y devuelve la lista de registros de una tabla JSON."""
    ruta = RUTAS[tabla]
    if not ruta.exists():
        inicializar_datos(seed=True)
    with ruta.open("r", encoding="utf-8") as f:
        try:
            datos = json.load(f)
        except json.JSONDecodeError:
            # Si está corrupto, lo reparamos a []
            datos = []
            _atomic_dump(ruta, datos)
    if not isinstance(datos, list):
        raise ValueError(f"El archivo {ruta.name} no contiene una lista JSON.")
    return datos
```

```python
def escribir(tabla: Tabla, registros: List[Dict[str, Any]]) -> None:
    """Sobrescribe por completo la tabla con la lista de registros proporcionada."""
    if not isinstance(registros, list):
        raise TypeError("Registros debe ser una lista de diccionarios.")
    ruta = RUTAS[tabla]
    _atomic_dump(ruta, registros)
```

```python
def eliminar_registro(tabla: Tabla, predicado) -> int:
    """Elimina los registros que cumplan el predicado y devuelve cuántos borró."""
    registros = leer(tabla)
    nuevos = [r for r in registros if not predicado(r)]
    borrados = len(registros) - len(nuevos)
    if borrados:
        escribir(tabla, nuevos)
    return borrados
```

## 7.2 Sistema de Menús Principales (main.py)

El archivo principal coordina la navegación entre módulos:

```python
def main():        You, last week • feat: Se creo la estructura y funcionalidad
    """Función principal del sistema"""

    # crea /datos y los .json si faltan
    inicializar_datos(seed=True)

    # Mensaje de bienvenida
    print("Iniciando Sistema de Biblioteca...")
    print("¡Bienvenido!")
    pausar()

    # Bucle principal del menú
    while True:
        mostrar_menu_principal()

        try:
            opcion = input("Seleccione una opción [1-4]: ").strip()

            if opcion == "1":
                menu_libros()
            elif opcion == "2":
                menu_usuarios()
            elif opcion == "3":
                menu_prestamos()
            elif opcion == "4":
                print("\n¡Gracias por usar nuestro sistema!")
                print("Hasta luego.")
                break
            else:
                print("\nOpción inválida. Por favor ingrese 1, 2, 3 o 4.")
                pausar()

        except KeyboardInterrupt:
            print("\n\n¡Hasta luego!")
            break
        except Exception as e:
            print(f"\nError: {e}")
            pausar()
```

### Funcionalidades:

- **Inicialización**: Carga datos al arranque con `inicializar_datos(seed=True)`
- **Navegación**: Menú principal con validación de opciones
- **Manejo de Errores**: Captura de interrupciones y excepciones
- **Integración**: Llama a los módulos especializados

```python
def mostrar_menu_principal():
    """Muestra el menú principal del sistema"""
    limpiar_pantalla()
    print("="*50)
    print("    SISTEMA DE GESTIÓN DE BIBLIOTECA")
    print("="*50)
    print("    Versión 1.0 - Grupo 5")
    print("    UADE - Programación I")
    print("="*50)
    print()
    print("1. GESTIÓN DE LIBROS")
    print("2. GESTIÓN DE USUARIOS")
    print("3. SISTEMA DE PRÉSTAMOS")
    print("4. SALIR DEL SISTEMA")
    print()
    print("="*50)
```
*

## 7.3 Módulo de Gestión de Libros (gestion_libros.py)

Implementa todas las operaciones CRUD para el catálogo de libros:

### Operaciones Implementadas:

- **Agregar Libro**: Validación de campos y generación automática de ID
- **Buscar Libro**: Búsqueda por título o autor (parcial o completa)
- **Listar Libros**: Visualización completa del catálogo
- **Eliminar Libro**: Eliminación con confirmación segura

```python
def agregar_libro():
    """Función para agregar un libro usando el sistema de persistencia"""
    print("\n--- AGREGAR LIBRO ---")

    # Validación de entrada
    while True:
        titulo = input("Título: ").strip()
        if titulo:
            break
        print("ERROR: El título no puede estar vacío.")

    while True:
        autor = input("Autor: ").strip()
        if autor:
            break
        print("ERROR: El autor no puede estar vacío.")

    # Obtener libros existentes para generar ID único
    libros_existentes = leer("libros")

    # Crear diccionario con ID único usando mejores prácticas
    libro = {
        "id": generar_id("LIB", libros_existentes, "id"),
        "titulo": titulo,
        "autor": autor
    }

    # Usar función de persistencia para guardar
    agregar_registro("libros", libro)
    print(f"\nLibro '{titulo}' agregado exitosamente!")
    print(f"ID asignado: {libro['id']}")
    pausar()
```

```python
def buscar_libro():
    """Función para buscar libros por título o autor"""
    limpiar_pantalla()
    print("--- BUSCAR LIBRO ---")

    print("Buscar por:")
    print("1. Título")
    print("2. Autor")
    print("3. Volver")

    while True:
        opcion = input("\nSeleccione opción [1-3]: ").strip()
        if opcion in ['1', '2', '3']:
            break
        print("ERROR: Seleccione 1, 2 o 3")

    if opcion == '3':
        return
```

### Características:

- **Validación de Entrada**: Verificación de campos obligatorios
- **Búsqueda Flexible**: Permite búsquedas parciales insensibles a mayúsculas
- **Confirmación Segura**: Requiere escribir 'CONFIRMAR' para eliminar

## 7.4 Módulo de Gestión de Usuarios (gestion_usuarios.py)

Maneja el registro y administración de socios de la biblioteca:

### Funcionalidades:

- **Registro de Usuario**: Captura de nombre, apellido y DNI
- **Buscar Usuario**: Por DNI exacto o nombre/apellido parcial
- **Listar Usuarios**: Visualización ordenada de todos los socios
- **Eliminar Usuario**: Con confirmación mediante 'CONFIRMAR'

```python
def buscar_usuario():        You, 43 minutes ago • Uncommitted changes
    """Función para buscar usuarios por DNI o nombre"""
    limpiar_pantalla()
    print("--- BUSCAR USUARIO ---")

    print("Buscar por:")
    print("1. DNI")
    print("2. Nombre o Apellido")
    print("3. Volver")

    while True:
        opcion = input("\nSeleccione opción [1-3]: ").strip()
        if opcion in ['1', '2', '3']:
            break
        print("ERROR: Seleccione 1, 2 o 3")

    if opcion == '3':
        return

    # Obtener término de búsqueda
    if opcion == '1':
        termino = input("Ingrese DNI: ").strip()
        campo = 'dni'
        busqueda_exacta = True
    else:
        termino = input("Ingrese nombre o apellido: ").strip().lower()
        campo = 'nombre_apellido'  # Búsqueda especial
        busqueda_exacta = False

    if not termino:
        print("ERROR: Debe ingresar un término de búsqueda")
        pausar()
        return
```

### Validaciones Implementadas:

- **DNI**: Verificación de 8 dígitos numéricos únicos
- **Nombres**: Validación de campos no vacíos
- **Duplicados**: Prevención de DNIs repetidos

## 7.5 Módulo de Sistema de Préstamos (gestion_prestamos.py)

Controla el flujo de préstamos de libros:

```python
def realizar_prestamo():        You, last week • feat: Se creo la estructura y fu
    """Funcion para realizar un prestamo usando el sistema de persistencia."""
    print("\n--- REALIZAR PRÉSTAMO ---")

    # Validacion de entrada
    while True:
        dni_usuario = input("DNI del usuario (solo digitos): ").strip()
        if dni_usuario and dni_usuario.isdigit():
            break
        elif dni_usuario:
            print("ERROR: El DNI debe contener solo digitos.")
        else:
            print("ERROR: El DNI del usuario debe contener 8 digitos.")

    while True:
        titulo_libro = input("Titulo del libro: ").strip()
        if titulo_libro:
            break
        print("ERROR: El titulo del libro no puede estar vacio.")

    # Obtener prestamos existentes para generar ID
    prestamos_existentes = leer("prestamos")

    # Crear diccionario con los datos usando las mejores practicas
    prestamo = {
        "id": generar_id("PREST", prestamos_existentes, "id"),
        "usuario_dni": dni_usuario,
        "libro_titulo": titulo_libro,
        "estado": "activo"
    }

    # Usar la función de persistencia para guardar
    agregar_registro("prestamos", prestamo)

    print(f"\nPrestamo registrado exitosamente!")
    print(f"ID del prestamo: {prestamo['id']}")
    pausar()
```

### Operaciones:

- **Realizar Préstamo**: Asocia usuario con libro y genera ID automático
- **Ver Préstamos Activos**: Lista todos los préstamos en curso
- **Buscar Préstamo**: Por DNI de usuario, título de libro o ID de préstamo
- **Eliminar Préstamo**: Para gestión de devoluciones

```python
def buscar_prestamo():        You, 40 minutes ago • Uncommitted changes
    """Función para buscar préstamos por DNI del usuario, título del libro o ID del préstamo"""
    limpiar_pantalla()
    print("--- BUSCAR PRÉSTAMO ---")

    print("Buscar por:")
    print("1. DNI del usuario")
    print("2. Título del libro")
    print("3. ID del préstamo")
    print("4. Volver")

    while True:
        opcion = input("\nSeleccione opción [1-4]: ").strip()
        if opcion in ['1', '2', '3', '4']:
            break
        print("ERROR: Seleccione 1, 2, 3 o 4")

    if opcion == '4':
        return

    # Obtener término de búsqueda
    if opcion == '1':
        termino = input("Ingrese DNI del usuario: ").strip()
        campo = 'usuario_dni'
        busqueda_exacta = True
    elif opcion == '2':
        termino = input("Ingrese título del libro (o parte): ").strip().lower()
        campo = 'libro_titulo'
        busqueda_exacta = False
    else:  # opcion == '3'
        termino = input("Ingrese ID del préstamo: ").strip()
        campo = 'id'
        busqueda_exacta = True

    if not termino:
        print("ERROR: Debe ingresar un término de búsqueda")
        pausar()
        return

    # Cargar préstamos y buscar
    prestamos = leer("prestamos")
    encontrados = []

    for prestamo in prestamos:
        valor_campo = str(prestamo.get(campo, ''))
```

### Estructura de Datos:

```
{
  "id": "PREST001",
  "usuario_dni": "12345678",
  "libro_titulo": "El Quijote",
  "estado": "activo"
}
```

## 7.6 Utilidades Compartidas (utilidades.py)

Funciones auxiliares utilizadas por todos los módulos:

```python
utilidades.py > ...
You, last week | 1 author (You)
"""        You, last week • feat: Se creo la estructura y funcionalidades b…
Utilidades del Sistema
Funciones auxiliares para el sistema de gestión de biblioteca
"""

import os

def limpiar_pantalla():
    """Limpia la pantalla de la consola"""
    os.system('cls' if os.name == 'nt' else 'clear')

def pausar():
    """Pausa la ejecución hasta que el usuario presione Enter"""
    input("\nPresione Enter para continuar...")

def mostrar_mensaje(mensaje, tipo="INFO"):
    """Muestra un mensaje con formato"""
    print(f"\n{mensaje}")
```

### Funciones:

- `limpiar_pantalla()`: Limpia la consola de forma multiplataforma
- `pausar()`: Pausa la ejecución esperando input del usuario
- `mostrar_mensaje()`: Muestra mensajes formateados

## 8 Validaciones adicionales implementadas

**Gestión de libros**:

- Verificación de que no se ingresen datos vacíos.
- Verificación de libro existente para evitar duplicados.

**Gestión de préstamos**

- Dni solicitado de al menos 7 dígitos.
- Validación de existencia de usuario antes de realizar préstamo.
- Validación de libro existente antes de realizar el préstamo.
- Validación de préstamo existente para evitar reservar libros ya prestados.

**Gestión de usuarios**

- Validación que nombre y apellido solo reciba letras al momento de registrar un usuario.
- Dni solicitado de al menos 7 dígitos.
- Validación que nombre y apellido solo reciba letras al momento de buscar un usuario.

## 9 Capturas del sistema funcionando

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

==================================================
    SISTEMA DE GESTIÓN DE BIBLIOTECA
==================================================
    Versión 1.0 - Grupo 5
    UADE - Programación I
==================================================

1. GESTIÓN DE LIBROS
2. GESTIÓN DE USUARIOS
3. SISTEMA DE PRÉSTAMOS
4. SALIR DEL SISTEMA

==================================================
Seleccione una opción [1-4]:
```

```
==========================================
      GESTIÓN DE LIBROS
==========================================
1. Agregar libro
2. Buscar libro
3. Listar todos los libros
4. Eliminar libro
5. Volver al menú principal
==========================================
Seleccione una opción [1-5]: 1

--- AGREGAR LIBRO ---
Título: Harry Potter y la piedra filosofal
Autor: JK Rowling

Libro 'Harry Potter y la piedra filosofal' agregado exitosamente!
ID asignado: LIB001

Presione Enter para continuar...
```

```
========================================
      GESTIÓN DE USUARIOS
========================================
1. Registrar usuario
2. Buscar usuario
3. Listar usuarios
4. Eliminar usuario
5. Volver al menú principal
========================================
Seleccione una opción [1-5]: 1

--- REGISTRAR USUARIO ---
Nombre: Facundo
Apellido: Tassone
DNI (solo dígitos): 38934487

Usuario 'Facundo Tassone' registrado exitosamente!
ID asignado: USR001

Presione Enter para continuar...
```

```
========================================
      SISTEMA DE PRÉSTAMOS
========================================
1. Realizar préstamo
2. Ver préstamos activos
3. Buscar préstamo
4. Eliminar préstamo
5. Volver al menú principal
========================================
Seleccione una opcion [1-5]: 1

--- REALIZAR PRÉSTAMO ---
DNI del usuario (solo digitos): 38934487
Titulo del libro: Harry Potter y la piedra filosofal

Prestamo registrado exitosamente!
ID del prestamo: PREST001

Presione Enter para continuar...
```

usuarios.json M

datos > usuarios.json > ...

You, 1 minute ago | 1 author (You)

```
[
  {
    "id": "USR001",
    "nombre": "Facundo",
    "apellido": "Tassone",
    "dni": "38934487"
  }
]
```

libros.json M

datos > libros.json > ...

You, 2 minutes ago | 1 author (You)

```
[        You, 2 minutes ago • Uncommitted changes
  {
    "id": "LIB001",
    "titulo": "Harry Potter y la piedra filosofal",
    "autor": "JK Rowling"
  }
]
```

prestamos.json M

datos > prestamos.json > ...

You, 35 seconds ago | 1 author (You)

```
[
  {
    "id": "PREST001",
    "usuario_dni": "38934487",
    "libro_titulo": "Harry Potter y la piedra filosofal",
    "estado": "activo"
  }
]
```